# Ders 7 Genişletilmiş Notu: Vize Öncesi Tekrar, SRS Belgesi Örneği, Etkinlik Şemaları ve Yazılım Kalitesi

## Ders 7 Genişletilmiş Notu: Vize Öncesi Tekrar, SRS Belgesi Örneği, Etkinlik Şemaları ve Yazılım Kalitesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Dönemin Başından Beri Neler Gördük?
- 2. CASE Araçları
- 3. Yazılım Geliştirme Süreç Modelleri
- 4. CMMI Süreç Sertifikasyonu
- 5. Gereksinim Mühendisliği ve UML Şemalarının Tekrarı
- 6. Tasarım Kalıbı Nedir?
- 7. Örnek SRS Belgesi: Etkinlik Duyuru Sistemi
- 8. Etkinlik Şemasının Sembolleri
- 9. Etkinlik Şeması Örnekleri
- 10. Tasarım Kalıplarının Tekrarı
- 11. Yazılım Kalite Yönetimi
- 12. Yazılım Kalite Özellikleri ve McCall'ın Bakış Açıları
- 13. Yeniden Kullanılabilirlik Nasıl Sağlanır?
- 14. Geliştirme Sürecinde Kaliteyi Sağlamak
- 15. Test Tekniklerine Kısa Giriş


## 1. Dönemin Başından Beri Neler Gördük?

Bu ders, vizeden önceki son konu dersiydi. Hoca dersin büyük kısmını o güne kadar işlenen konuların tekrarına ayırdı. Tekrarı yaparken bazı konuları yeni örneklerle pekiştirdi: önceki yıllarda öğrencilerin hazırladığı bir SRS belgesi ve kulvarlı iki etkinlik şeması. Dersin sonunda da vizeden sonra genişletilecek yeni bir konuya, yazılım kalitesine giriş yaptı.

Hoca tekrara ders planı üzerinden gitti. İlk hafta dersin içeriği tanıtılmış, yazılım mühendisliği kavramı ve yazılımı etkileyen genel bilgiler üzerinde durulmuştu. O hafta şu sorular ele alınmıştı:

- Yazılım nedir?
- Yazılım mühendisliği nedir?
- Yazılım süreç modeli nedir?
- Yazılım geliştirmenin alt başlıkları nelerdir?
- CASE kavramı nedir?

## 2. CASE Araçları

Hoca CASE kavramını bir soruyla hatırlattı: Staj yaptığınız yerde "CASE kullanacağız" dendiğinde, "CASE nedir?" sorusuna ne cevap verirsiniz?

Bir öğrenci CASE'in "bilgisayar destekli yazılım mühendisliği" (Computer-Aided Software Engineering) demek olduğunu söyledi. Bir cümleyle tanımlaması istenince de bunların UML diyagramlarını program yardımıyla oluşturmak için kullanılan programlar olduğunu söyledi.

Hoca bu cevabın yalnız analiz ve tasarım aşamasındaki desteği anlattığını belirtti. Bu destek, UML diyagramlarının hem daha hızlı ve otomatik çizilmesini hem de kontrol edilmesini içerir. Ama CASE araçları bununla sınırlı değildir. Yazılım geliştirme sırasında kodlamada, konfigürasyon yönetiminde ve test aşamasında (kullanıcı testi, bütünleştirme testi, sistem testi) da yardımcı yazılım araçları kullanılır.

Bu çerçevede hocanın tanımı şudur: CASE araçları, yazılım geliştirme sürecinin her evresinde yazılım geliştiricilere destek olan, işlerini hızlandıran ve daha kontrollü yapılmasını sağlayan yazılımlardır.

CASE araçları iki biçimde karşımıza çıkar:

- **Bütünleşik (integrated) CASE araçları:** Sürecin birçok aşamasını birlikte destekleyen büyük araçlar.
- **Küçük CASE araçları:** Yalnız belli bir aşamaya destek veren araçlar.

Bütün yazılımlarda olduğu gibi CASE araçlarının da açık kaynaklı olanları vardır. Ama büyük yazılım projelerinde kullanılan profesyonel araçlar ücretlidir ve çok pahalıdır.

## 3. Yazılım Geliştirme Süreç Modelleri

İkinci haftada yazılım geliştirme süreç modelleri işlenmişti. Önce klasik süreç modelleri görülmüştü:

- **Şelale (waterfall) modeli.**
- **Prototip modeli:** Türkçede "ön ürün" diye de karşılanabilir.
- **RAD (Rapid Application Development):** Hızlı uygulama geliştirme.
- **Bileşen tabanlı (component based) yaklaşım.**
- **Spiral model.**
- **Artımlı (incremental) yaklaşım:** Hoca bunu spiral modelin artımlı yapısının ilk örneği olarak tanıttı.

Klasik modellerin yanında Unified Process modeli de işlenmişti. Bu model, IBM'in sonradan satın aldığı ve bugün IBM'in bir platformu olarak sunulan Rational firmasının RUP (Rational Unified Process) yapısına dayanır. Hoca bu modelin özellikle geçiş sürecinde büyük firmalar tarafından çok kullanıldığını söyledi.

Son olarak çevik (agile) süreçler görülmüştü. Çevik modeller önce orta ve küçük boy yazılım şirketlerinde kullanılmaya başlandı. Bugün büyük askerî yazılımlarda bile kullanılıyor. Çevik süreçler içinde Kanban da vardır, ama derste XP (Extreme Programming) ve Scrum üzerinde durulmuştu. Hoca süreç modellerinin şekillerinden oluşan bir özet slaytı da paylaşacağını söyledi.

## 4. CMMI Süreç Sertifikasyonu

Süreç modellerinden sonra, süreçleri ve proje yönetimini iyileştirmeye yönelik bir sertifikasyon konuşulmuştu. Bu sertifikasyon, yazılım geliştirme ortamını (yazılım evini ya da yazılım geliştirme birimini) süreçlerine, proje yönetimine ve hatta çalışanlarla ilgili olanaklarına göre değerlendirip derecelendirir. Bu sertifikasyonun adı CMMI'dır.

Hoca CMMI'nın açılımını verdi ve Türkçesini "bütünleşik yetkinlik olgunluk modeli" olarak söyledi.

> **Ders sonrası not:** Altyazıda açılım "Capability Majority Model Integration" diye geçiyor. Doğrusu büyük olasılıkla **Capability Maturity Model Integration**'dır. Türkçe karşılığı da buna uygun olarak "bütünleşik yetkinlik olgunluk modeli"dir.

### 4.1 Öğrencilerin hatırladıkları

Hoca CMMI'nın ne olduğunu, kaç düzeyi olduğunu ve hangi sertifikaların alınabildiğini sordu. Bir öğrenci altı düzey olduğunu hatırladığını ama açıklayamayacağını söyledi. Başka bir öğrenci düzeyleri şöyle hatırladı: Sıfırıncı düzeyde henüz bir şey yapılmıyor. Birinci düzeyde işler yürütülmeye başlanıyor. İkinci düzeyde asıl yönetim başlıyor ve SRS belgesi hazırlanıyor. Hoca bu cevapları düzeylerin nasıl oluştuğunu anlatarak tamamladı.

### 4.2 Düzeylerin anlamı

Hoca beş düzey olduğunu söyledi ve düzeyleri ikinci düzeyden itibaren anlattı.

**İkinci düzey:** Bir yazılım şirketi ikinci düzeydedir dediğimizde şunu kastederiz:

- Gerçekleştirdikleri projelerde gereksinim analizi yaparlar.
- Gereksinim analizinin sonunda SRS belgesi, tasarımın sonunda tasarım belgesi oluştururlar. Çalışmaları bu anlamda kontrol altındadır.
- Proje planı yaparlar.
- Konfigürasyon yönetimi uygularlar.

Hoca burada yazılım konfigürasyon yönetiminden ne kastedildiğini açıkladı. Yazılımın her aşamasında ürünler elde edilir. Bu ürünlerin yapısıyla ilgili bir veri tabanı oluşturulur. Bu veri tabanı kullanılan platformla otomatik olarak da oluşturulabilir, bu iş için geliştirilmiş bir yazılımla da. Amaç yazılım elemanlarını takip etmektir. Yani her sürümde (version) hangi yazılım elemanının hangi durumda olduğunu gösteren bir tanım tutulur. İkinci düzeyden itibaren buna çaba gösterilir.

**Üçüncü düzey:** Burada bir adım daha ileri gidilir.

- Yazılım biriminde risk yönetimi yapılır.
- Bitmiş bütün projelerde aynı yapıda bir gereksinim belgesi bulunur. Yani mesele yalnız gereksinim belgesi tutmak değildir. Bu belgenin bütün projelerde aynı şekilde takip edilmesi gerekir.
- Böylece şirket genelinde bir ortaklık oluşur: Bütün projelerde aynı seviyede, aynı belgeler hazırlanır.

### 4.3 CMMI'nın gelişimi

Hocaya göre CMMI, 2000'li yılların başında proje ile ilişkili teknik yazılım geliştirme süreçlerini ve proje yönetimini birleştirdiği için "bütünleşik" adını almıştı. 2018'den sonraki çalışmalarla bu bütünleşik yapıya insan kaynağı konusundaki yetkinlikler de eklendi: projelere yazılım geliştirici alımı, geliştiricilerin yeteneklerinin kullanımı, kariyer planlarının yapılması. Şirket bunları da aynı aşamalarda sağlıyorsa bütünleşik düzeyde üçüncü ya da dördüncü düzeye çıkabiliyor.

> **Ders sonrası not:** Hocanın "2018" dediği gelişme, büyük olasılıkla 2018'de yayımlanan CMMI V2.0 sürümüdür. Bu sürümde insan kaynağı ve iş gücü yönetimiyle ilgili uygulama alanları modele dahil edildi.

### 4.4 Türkiye'de CMMI

Hoca Türkiye'de CMMI sertifikası alan firmaları sordu. Bir öğrenci Koç Sistem ve İTİA'yı saydı. Hoca bunları onayladı ve askerî yazılım üreten Milsoft'un sürekli beşinci düzeyde olan bir firma olduğunu ekledi. Hocaya göre bu alanda yazılım üreten diğer firmalar da en az dördüncü düzeydedir. Ayrıca bu alanda kullanılacak yazılımları üreten firmaların en az üçüncü düzeyde olması gibi bir şart da vardır.

> **Ders sonrası not:** CMMI'nın aşamalı (staged) gösteriminde büyük olasılıkla olgunluk düzeyleri (maturity levels) 1'den 5'e kadardır: 1 Başlangıç (Initial), 2 Yönetilen (Managed), 3 Tanımlı (Defined), 4 Nicel Olarak Yönetilen (Quantitatively Managed), 5 Optimize Eden (Optimizing). Öğrencinin andığı "sıfırıncı düzey" olgunluk düzeyi değildir; sıfırdan başlayan numaralandırma, süreç alanlarının yetkinlik düzeylerinde (capability levels) kullanılır.

## 5. Gereksinim Mühendisliği ve UML Şemalarının Tekrarı

Dördüncü haftada gereksinim mühendisliği (requirements engineering) işlenmişti. Hoca bu derste konunun kafada daha iyi şekillenmesi için önceki yıllarda öğrencilerin hazırladığı bir SRS belgesini inceleyeceklerini söyledi (Bölüm 7).

SRS belgesi ve diğer bütün belgeler için yazılım mühendisliği standartları vardır. Bu standartlar, yazılım geliştirme sürecinde elde edilen belgelerin biçimini (formatını) gösterir. Derste gereksinim analizi sonunda üretilen belgenin formatı üzerinde durulacaktı.

Hoca o güne kadar görülen modelleme konularını da saydı:

- **Kullanım senaryoları (use case):** Hem metinleri (belgeleri) hem de diyagramları görülmüştü.
- **Sınıf şeması (class diagram):** Nesneye yönelik programlama dersinde de görülen UML diyagramı. Bu dersi almamış öğrenciler için yeniden işlenmişti. Tasarım sürecinde UML sınıf şeması üzerinde durulmuştu.
- **Sıralı (ardışıl) diyagramlar (sequence diagram).**
- **Etkinlik (activity) ve durum (state) şemaları.**

Hoca sıralı diyagramların ve sınıf diyagramlarının başka derslerde de çok kullanıldığını, ama etkinlik şemasının daha az görüldüğünü söyledi. Bu yüzden etkinlik şeması için bu derste bir örnek daha yapılacaktı (Bölüm 9).

## 6. Tasarım Kalıbı Nedir?

Geçen hafta tasarım kalıpları (design patterns) işlenmişti. Hoca bunları “Gang of Four” (GoF) kalıpları olarak, yani dört yazılım uzmanı tarafından geliştirilmiş kalıplar olarak hatırlattı.

Hoca “Tasarım kalıbı dendiğinde ne anlıyorsunuz, neden kullanırız?” diye sordu. Bu sırada bir ipucu verdi: Tasarım kalıpları yeniden kullanılabilirliği (reusability) sağlayan en önemli etkenlerden biridir.

Bir öğrenci, tam hatırlamadığını belirterek, tasarım kalıbının sınıf tasarımlarında daha önce yapılmış ve verimliliği kanıtlanmış bazı kalıpların kullanılması olduğunu söyledi.

Hoca bu cevabı genişletti. Sınıf diyagramlarında, sıralı diyagramlarda ve sınıflar üzerinde yapılacak işlemlerde daha önce çözümü bulunmuş ve tanımlanmış algoritmalar ve yapılar vardır. Bunların hazır hâle getirilip sürekli kullanılacak biçimde saklanmasına tasarım kalıbı denir.

Hoca bunu “aklın yolu birdir” sözüyle açıkladı: Bir kez uğraşıp çözüm bulduğumuz bir problemle tekrar tekrar uğraşmayalım. Bu çözüm, aynı problemle karşılaşanların kullanabileceği bir kalıp olarak saklansın ve sürekli kullanılsın.

Hoca ayrıca bu dört uzmanın ilk tanımladığı temel kalıplardan en çok kullanılan dokuzunun geçen hafta işlendiğini söyledi. Bunların birkaçına “Sistem Analizi ve Tasarımı” kitabından örnek de verilmişti.

> **Ders sonrası not:** Hoca burada kalıpların ilk tanımlamada 26 tane olduğunu ve 2000’li yılların başına ait olduğunu söyledi. Dersin ilerleyen kısmında sayıyı kendisi 23 olarak düzeltti. Doğrusu büyük olasılıkla şudur: Gamma, Helm, Johnson ve Vlissides 1994’te yayımladıkları *Design Patterns* kitabında **23** kalıp tanımladı.

## 7. Örnek SRS Belgesi: Etkinlik Duyuru Sistemi

### 7.1 SRS nedir?

SRS, Software Requirements Specification, yani yazılım gereksinimleri belirtimi demektir. Hoca “spesifikasyon”un ayrıntılı açıklama anlamına geldiğini hatırlattı. SRS belgesi, yazılım gereksinimlerinin ayrıntılı açıklamasıdır. Bu belgede gereksinim analizinde yapılan çalışmaların belgelenmiş hâli bulunur.

İncelenen belge, “Etkinlik Duyuru Sistemi” adlı bir proje için hazırlanmıştı. Belgenin içerik listesi, IEEE yazılım standartları içinde 830 numaralı standartta belirtilen içeriğe göre oluşturulmuştu. Hoca IEEE 830’un ilk hâlinin 1998’de oluşturulduğunu ve yazılım gereksinimleri belirtimi için önerilen yapı olarak tanımlanabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** IEEE 830-1998 standardının yerini büyük olasılıkla 2011'de ISO/IEC/IEEE 29148 standardı aldı ve bu standart 2018'de güncellendi. Yine de 830'un bölüm yapısı ders projelerinde ve birçok kurumda hâlâ yaygın olarak kullanılır.

### 7.2 Sistemin kapsamı

Etkinlik duyuru sistemi, etkinliklerin duyurulduğu bir sitedir. Siteye bilgi girişini idari personel yapar. Belgedeki blok diyagrama göre sistemde şu işlemler bulunur:

- Etkinlik arama.
- Etkinlik listeleme.
- Etkinliğin ayrıntılarını görüntüleme.
- Bazı etkinlikler için satın alma.

### 7.3 Belgenin bölümleri

Hoca belgenin bölümlerini sırayla gösterdi.

**1. Giriş:** Resmî bir giriş bölümüdür. Alt başlıkları şunlardır:

- Amaç.
- Kapsam: "Etkinlik duyuru sistemi dendiğinde ne anlıyoruz?" sorusunun cevabı.
- Tanımlar ve kısaltmalar: SRS belgesi içinde kullanılacak kısaltmaların listesi.
- Referanslar: Bu belgede IEEE 830 standardı referans olarak verilmişti.
- Belgeye genel bakış: Belgede hangi bölümlerin bulunduğu.

**2. Genel tanım:** Ürünün hangi alt parçalardan oluştuğu ve bu alt parçalara göre kullanıcı isteklerinin tanımları yazılı olarak verilir.

- Ürüne bakış: Bir blok diyagramla sistemde hangi özelliklerin olacağı ve çalışma sırası gösterilir.
- Ürün işlevleri: Üründe hangi işlemlerin gerçekleştirileceği.
- Arayüzler: Kullanıcı arayüzleri, donanım arayüzleri ve yazılım arayüzleri tanımlanır.

Hoca bu kısımda işlevsel (fonksiyonel) gereksinimlerin sıralanmaya başladığını gösterdi. Yazılımda olması gereken işlemler bir numaralandırma (kodlama) düzeniyle sıralanır.

**3. Özel gereksinimler:**

- **İşlevsel gereksinimler (functional requirements):** Örneğin etkinlik ekleme, silme, güncelleme ve listeleme işlemleri. Bunlara ek olarak kullanım şemaları (use case diagram) da verilebilir. UML kullanım şemalarında gereksinim analizinin modellemesi zaten bulunur.
- **Performans gereksinimleri:** Hoca bunların işlevsel olmayan gereksinimler (non-functional requirements) olduğunu belirtti.
- **ER diyagramları.**
- **Tasarım kısıtları:** Hangi tasarım kısıtlarının olacağı.
- **Kalite özellikleri:** Bunlar da işlevsel olmayan gereksinimlerdir. Hoca bu başlığa "işlevsel olmayan gereksinimler" adının da verilebileceğini, performans gereksinimlerinin de bunun içinde yer alabileceğini söyledi.
- **Gereksinimlerin izlenebilirliği (traceability):** İzlenebilirliğin nasıl sağlanacağı.

**Ekler:** Ekran yapıları (ekran tasarımları), kullanım senaryosu tanım şablonları ve bir kullanım şeması.

### 7.4 SRS belgesinde kullanım senaryosu belgeleri

Hoca SRS belgesinde kullanım senaryosu (use case) belgelerinin bulunması gerektiğini vurguladı. İşlevsel gereksinimler madde madde, 1.1.1, 1.1.2, 1.1.3 gibi numaralarla yazılır. Bunun yanında kullanım senaryosu belgeleri de eklenir. Böylece her gereksinimin hangi grup içinde ve nasıl tanımlandığı gösterilir.

> **Hocanın vurgusu:** Her bir gereksinim bloğu (işlev) için bir kullanım senaryosu belgesi oluşturmak gerekir.

Örnek belgedeki kullanım senaryosu belgesinde şu alanlar vardı:

- Kullanım senaryosunun numarası ve adı.
- Hazırlayan (kimin çizdiği).
- Aktörler.
- Birincil aktör: Hoca bunun da açıklanması gerektiğini, ama örnek belgede açıklanmadığını belirtti.
- Ön koşul: İşin başlaması için gereken koşul.
- Son koşul: İşin bittiğini belirten koşul.
- Kullanım sıklığı ve önceliği: Hoca bu alanın kendi derslerindeki örneklerde bulunmadığını, bu belgede ayrıca tanımlandığını söyledi.
- Ana akış: Sistemin algoritmasının düzenli gittiği durum. Bu örnekte 12 adım vardı.
- Alternatif akış: Ana akıştaki adımlardan herhangi birinde bir durma ya da problem olursa ne yapılacağı ve nereye dönüleceği.

## 8. Etkinlik Şemasının Sembolleri

Hoca etkinlik şemalarıyla (activity diagram) ilgili yeni bir slayt paylaştı ve önce sembolleri hatırlattı:

- **Etkinlik (eylem):** Hoca her etkinliğin bir elipsle gösterildiğini söyledi.
- **Başlangıç noktası:** İçi dolu bir daire.
- **Bitiş noktası:** Bir daire (halka) içindeki içi dolu daire.
- **Kulvar başlığı:** İşin hangi kullanıcı grubu tarafından yapıldığını belirten üst başlık.
- **Senkronizasyon çubuğu:** Etkinlik şemalarında bulunur. İki kullanımı vardır:
  - **Çatal (fork, split):** Bir noktadan itibaren tek akışla giden işlemin iki ayrı kolda gerçekleştiğini gösterir.
  - **Birleşme (join, kaynaştırma):** İki ayrı kolda giden işlemlerin birleştiğini ve bundan sonra tek akışla devam edeceğini gösterir.
- **Karar:** Eşkenar dörtgenle gösterilir. Karar verilmesi gereken noktada eşkenar dörtgenin üzerine karar cümlesi yazılır. Hoca bazen karar mekanizmasının elips üzerinden de gösterilebildiğini söyledi.

  **Ders sonrası not:** UML etkinlik şemasında eylemler (etkinlikler) büyük olasılıkla kenarları yuvarlatılmış dikdörtgenle çizilir. Elips, kullanım şemasında kullanım senaryosunu gösteren semboldür. Karar düğümü için de UML'de eşkenar dörtgen kullanılır.

## 9. Etkinlik Şeması Örnekleri

Hoca etkinlik şemalarının bu yönünü en iyi anlatanın iki örnek olduğunu söyledi. İki örnek aynı konunun biraz farklılaştırılmış hâlleriydi.

Hoca etkinlik şemasının neden önemli olduğunu şöyle açıkladı: Modelleme yaparken hangi kullanıcıların hangi işleri yaptığını belirleriz. Bunu oluşturduktan sonra, o işle ilgili ayrıntıyı almak için soru sormak üzere o kullanıcıya gidebiliriz. Bir sistemde insan kullanıcıların dışında bir donanım sistemi ya da yazılım sistemi de nesneler arasında yer alabilir.

### 9.1 Örnek 1: Teklif (kota) hazırlama

Bir satış işleminde dört taraf vardır: müşteri, satıcı, teknik uzman ve sistemde gerçekleştirilen işlemler. Bunların her biri ayrı bir kulvardır. Altyazıda "kota" diye geçen kelime, müşteriye verilen fiyat teklifidir (quote).

Akış şöyledir:

1. Süreç müşterinin gelmesiyle başlar. Müşteri bir teklif ister.
2. Satıcı müşterinin gereksinimlerini not alır ve teknik destek isteyip istemediğini sorar.
3. Teknik destek gerekiyorsa müşteri teknik uzmana yönlendirilir. Teknik uzman gereksinimleri değerlendirir, gereksinimlerle ilgili bilgileri sisteme girer ve teklifi hesaplar.
4. Teknik destek gerekmiyorsa satıcı bilgileri sisteme kendisi girer ve teklifi hesaplar.

5. İki yoldan hangisiyle gelinirse gelinsin, yani teknik uzmana uğransın ya da uğranmasın, hesaplamadan sonra müşteriye dönülür ve müşteriden teklifi gözden geçirmesi istenir.
6. Müşteri değişiklik istiyorsa en başa dönülür.
7. Değişiklik istemiyorsa siparişi kabul eder ve işlem sonlanır.

Metin olarak şema şöyle özetlenebilir:

```
Kulvarlar: Müşteri | Satıcı | Teknik Uzman | Sistem

(başla)
Müşteri:      teklif ister
Satıcı:       gereksinimleri not alır
Satıcı:       <teknik destek gerekli mi?>
   [evet]  -> Teknik Uzman: gereksinimleri değerlendirir
              Teknik Uzman: bilgileri sisteme girer
              Sistem:       teklifi hesaplar
   [hayır] -> Satıcı:       bilgileri sisteme girer
              Sistem:       teklifi hesaplar
Müşteri:      teklifi gözden geçirir
Müşteri:      <değişiklik istiyor mu?>
   [evet]  -> başa dön
   [hayır] -> siparişi kabul eder
(bitir)
```

> **Hocanın vurgusu:** Bu şemada asıl gösterilmek istenen, farklı bakış açılarına ve farklı kullanıcı taraflarına göre hangi tarafta hangi işlemlerin yapıldığıdır. Kullanım şemasından farklı olarak etkinlik şeması, her kullanıcı grubunun yaptığı işi ayrı bir alanda göstererek bu farkı daha iyi anlatır.

### 9.2 Örnek 2: Sipariş alma

İkinci örnek bir sipariş alma işlemidir. Kulvarlar satış, mühendislik, satın alma, üretim ve planlama (scheduling) taraflarıdır.

1. Süreci satıcı başlatır: Siparişi kabul eder ve siparişi oluşturur.
2. Mühendislik tarafındaki kişiler siparişte belirtilenlere göre ne istendiğini, yani spesifikasyonları oluşturur.
3. Ardından akış ikiye ayrılır: Satın alma tarafı gerekli ürünleri (malzemeleri) satın alır, üretim tarafı da üretimi gerçekleştirir.
4. Her iki taraftaki işlemler bitince akış birleşir. Planlama tarafında üretim planı (schedule) oluşturulur ve işlem sonlanır.

```
Kulvarlar: Satış | Mühendislik | Satın Alma | Üretim | Planlama

(başla)
Satış:        siparişi kabul eder
Satış:        siparişi oluşturur
Mühendislik:  spesifikasyonları oluşturur
==== ÇATAL (fork) ====
   Satın Alma:  malzemeleri satın alır
   Üretim:      üretimi gerçekleştirir
==== BİRLEŞME (join) ====
Planlama:     üretim planını (schedule) oluşturur
(bitir)
```

### 9.3 Etkinlik şeması süreçte ne zaman çizilir?

Hoca etkinlik şemasının kullanım şemasından başlayarak çizildiğini söyledi. Etkinlik şeması, kullanım şemasının biraz daha açıklanmış, gelişmiş hâlidir. Yani gereksinim analizinden bir adım sonra, tasarıma geçiş aşamasında çizilir. Böylece sistemin hangi kullanıcı çerçevesinde hangi işlemleri yaptığı daha rahat gösterilir.

Bu aşamada eksik bilgiyi tamamlamak için farklı kullanıcı tiplerine kapalı uçlu ve açık uçlu sorular sorulabilir ya da anketler dağıtılabilir.

## 10. Tasarım Kalıplarının Tekrarı

Aradan sonra hoca, geçen hafta öğrencilerden bu konuda yeterli geri dönüş alamadığını söyleyerek tasarım kalıplarına yeniden döndü. Önce sayıyı düzeltti: Daha önce 26 demişti, ama Gamma, Helm, Johnson ve Vlissides tarafından ortaya konan kalıpların sayısı 23'tür. Ardından geçen hafta işlenen dokuz kalıbı ders kitabından tek tek hatırlattı:

- **Yaratıcı (Creator):** Bir sınıftan nesne yaratma sorumluluğunun hangi sınıfa verileceğini belirler.
- **Bilginin uzmanı (Information Expert):** Nesnelere sorumluluk atamanın temel ilkesidir. Hangi sınıfa hangi işin, hangi mesajla ilgili işlemin atanacağı sorusuna çözüm getirir.
- **Az bağımlılık (Low Coupling):** Bir sınıfın diğer sınıflardaki değişimlerden en az etkilenmesini sağlar. Sınıflar birbirinden bağımsız oldukça yeniden kullanılabilirlik de artar.
- **Denetçi (Controller):** Sistem olaylarını alan ve kontrol eden sınıf.
- **İyi uyum (High Cohesion):** Bir sınıfın (nesnenin) kendi sorumluluklarına yoğunlaşması ve onları iyi yapabilmesidir. Bir sınıf çok fazla iş yapıyorsa sorumluluklar dağınıktır; bu sınıfta uyum kötüdür ve yeniden kullanımı da zorlaşır. Hoca uyumun, tasarımın kalitesine bakarken üzerinde durulan işlerden biri olduğunu ve burada bir tasarım kalıbı olarak da tanımlandığını söyledi.
- **Çok şekillilik (Polymorphism):** Birbiriyle ilgili ama tiplere göre değişen davranışları tasarlarken çok şekilli metotların kullanılması.
- **Yapay sınıf (Pure Fabrication):** Uzman kalıbının getirdiği çözümler iyi uyum ve az bağımlılık kavramlarıyla çelişiyorsa yapay sınıflar kullanılır.
- **Dolaylılık, ara birim (Indirection):** Kolayca değişebilecek yapılar arasındaki bağımlılığı azaltmak gerekir, çünkü bu yeniden kullanılabilirliği artırır. Dolaylılık bunu sağlayan yapıdır ve sınıfların dış değişimlerden etkilenmesini azaltır.
- **Değişimlerden korunma (Protected Variations):** Hocanın deyişiyle dikkat edilmesi gereken tasarım ilkelerinin sonuncusu. Bir sistemdeki her alt sistemde kararsızlık olabilir. Bu kararsızlığın diğer birimlere etkisini en aza indirmek için yapılan işlemlerdir. Daha kararlı bir sistem kurmanın gereğidir.

  **Ders sonrası not:** Hoca yapay sınıfı anlatırken "gerçek dünyadaki varlıklarla benzerliği olan, bunlara göre yaratılan yapay sınıflar" dedi. Doğrusu büyük olasılıkla tersidir: Yapay sınıf (Pure Fabrication), problem alanında (gerçek dünyada) karşılığı olmayan, yalnız uyumu artırmak ve bağımlılığı azaltmak için uydurulan bir sınıftır. Örneğin veri tabanı işlemlerini üstlenen bir kalıcılık (persistence) sınıfı.

  **Ek bilgi (derste anlatılmadı):** Derste sayılan dokuz kalıp büyük olasılıkla GoF'un 23 kalıbı değildir. Bunlar Craig Larman'ın sorumluluk atama ilkeleri olan GRASP (General Responsibility Assignment Software Patterns) kalıplarıdır. GoF kalıpları ise Singleton, Factory Method, Observer, Adapter ve Strategy gibi kalıplardır. İki grup da nesneye yönelik tasarımda kullanılır, ama GRASP daha çok "hangi sorumluluk hangi sınıfa verilmeli?" sorusuna yöneliktir.

Hoca geçen hafta bu kalıpların bir etkileşim diyagramında nasıl kullanılacağı üzerinde de durulduğunu hatırlattı. Etkileşim diyagramı (interaction diagram) genel bir addır; işbirliği diyagramları (collaboration diagram) da bu gruptadır. Ama derste anlaşılan anlamıyla etkileşim diyagramı, sıralı diyagramdır (sequence diagram). Bu diyagramda nesneler en üstte işleme giriş sırasıyla tanımlanır. Altta da aynı sırayla, belli bir noktaya gelene kadar hangi işlemlerin yapıldığı ve bundan sonra hangi işlemlerin yapılacağı gösterilir.

Hoca böylece vize öncesi tekrarın bittiğini söyledi.

## 11. Yazılım Kalite Yönetimi

Dersin son kısmında, sonraki haftalarda genişletilecek bir konuya, yazılım kalitesine giriş yapıldı. Hoca son sınıfta "kalite ve test teknikleri" adlı seçmeli bir ders de olduğunu söyledi. O derste önce kalite ve kalite yönetimi, sonra test teknikleri işlenir. Test, kalite yönetimi içindeki

işlemlerin en kapsamlısıdır. Test aslında yazılım geliştirme sürecinin adımlarından biridir, ama ayrı bir bölüm olarak ele alınır.

Hoca eski bir sunumdan bir slayt paylaştı. Yazılım kalite yönetimi denince şu sorular akla gelir:

- Yazılım kalite özellikleri nelerdir?
- Yazılım kalitesi nasıl ölçülür?
- Yazılım kalitesi nasıl sağlanır?
- Yazılımın test edilmesi (sınanması) nerede yer alır? Yazılımın test edilmesi, kaliteyi sağlamak için yapılan işlerden biridir.

Kalite ölçümü, kalitenin sağlanması ve kalitenin sınanması sonraki haftalarda işlenecekti. Bu derste yalnız kalite özellikleri anlatıldı. Hoca kalite özelliklerinin, yazılımla ilgili genel bilgiler sırasında da görülebilecek kadar genel bir konu olduğunu söyledi.

## 12. Yazılım Kalite Özellikleri ve McCall'ın Bakış Açıları

Yazılım kalite özellikleri, bir yazılıma "kaliteli" diyebilmek için hangi özelliklere sahip olmasını istediğimizi gösterir. Hoca bu özellikleri McCall'ın tanımlamasına göre, farklı bakış açılarıyla değerlendirdi. Hocanın saydığı üç bakış açısı şunlardı:

1. Kullanıcı bakış açısı.
2. Daha sonra bakım yapacak yazılım uzmanlarının bakış açısı.
3. Yazılımın kullanımı sırasında işletimini sağlayan kişilerin bakış açısı.

Hoca biraz sonra aynı grupları "kullanıcı, çalıştırıcı ve değişiklik yapan bakış açısı" olarak da adlandırdı.

### 12.1 Özellikler tek tek

- **Doğruluk (correctness):** Yazılımdan ne beklediğimizi gereksinim spesifikasyonunda belirttik. Yazılım bu spesifikasyonlara uyarak müşteri isteklerini ne kadar karşılıyorsa, yani denilenleri ne kadar yapıyorsa o kadar doğru çalışıyordur.
- **Güvenilirlik (reliability):** Tasarlanan işlevleri istenen duyarlılıkta yerine getirme olanağıdır. Hocanın örneği bir muhasebe yazılımıdır: Yazılım belli işlemleri doğru olarak yapıyor, ama bu doğruluk yüzde kaç güvenilirlikle sağlanıyor? Yüzde 99 mu, yüzde 99,8 mi? Güvenilirlik, yazılımcının hazırladığı yazılıma ne kadar güvenildiğini, elde edilen sonucun yüzde kaç doğru olduğunu gösterir. Hoca bunun doğruluktan da güvenlikten de farklı bir şey olduğunu, yalnız kelime benzerliği bulunduğunu vurguladı.
- **Bağlanabilirlik, birlikte çalışabilirlik (interoperability):** Bir sistemin diğerine bağlanabilme olanağı. İki sistemin birlikte çalışabilme kolaylığı ve birbiriyle veri alışverişini nasıl yapacağı.
- **Verimlilik (efficiency):** Programın işlevlerini yerine getirebilmesi için gereken bilgi işlem kaynakları ve kodlamanın gideri.
- **Güvenlik (integrity, security):** Yetkisiz kişilerin yazılıma erişimini ve veri girişini önleme olanağı.
- **Kullanışlılık (usability):** Öğrenme, işletme, girdi hazırlama ve çıktı yorumlama kolaylığının derecesi. Hoca bunu kullanıcı dostu olmak diye de özetledi.
- **Hata bulma kolaylığı (maintainability).**
- **Esneklik (flexibility).**
- **Test etme kolaylığı (testability).**
- **Taşınabilirlik (portability):** Programın farklı donanımlarda ve değişik yazılım sistemi ortamlarında kullanılabilme olanağı.
- **Yeniden kullanılabilirlik (reusability):** Programın tamamının veya bir bölümünün başka bir uygulamada kullanılabilme olanağı.

### 12.2 Hocanın gruplaması

Hoca özellikleri şöyle grupladı:

- **Kullanıcı için önemli olanlar:** İlk altı özellik, yani doğruluk, güvenilirlik, birlikte çalışabilirlik, verimlilik, güvenlik ve kullanışlılık.

- **Düzeltici bakış açısıyla önemli olanlar:** Hata bulma kolaylığı, esneklik, test etme kolaylığı.
- **Değiştirici bakış açısıyla önemli olanlar:** Taşınabilirlik ve yeniden kullanılabilirlik.

Hoca son iki özellik için şunu ekledi: Burada hem yazılımı geliştirene ne kadar güvendiğiniz hem de gerekli küçük değişiklikleri sonradan kendiniz yapıp yapamayacağınız önem kazanır.

> **Ders sonrası not:** McCall modelinde üç grup büyük olasılıkla şöyledir. Ürünün işletimi (product operation): doğruluk, güvenilirlik, verimlilik, bütünlük (integrity) ve kullanışlılık. Ürünün gözden geçirilmesi (product revision): bakım kolaylığı, esneklik ve test edilebilirlik. Ürünün geçişi (product transition): taşınabilirlik, yeniden kullanılabilirlik ve birlikte çalışabilirlik. Yani hocanın kullanıcı grubuna koyduğu birlikte çalışabilirlik, McCall'da geçiş grubundadır.

### 12.3 ISO standartlarındaki yaklaşım

Hoca kaliteli yazılım özelliklerinin ISO standartlarında da (IEEE içinde de benzer bir standart vardır) verildiğini söyledi. ISO standartlarında benzer özellikler bulunur, ama kullanıcı, çalıştırıcı ve düzeltici bakış açısı gibi bir sınıflama yapılmadan hepsi tek liste hâlinde sıralanır.

> **Ek bilgi (derste anlatılmadı):** Güncel ISO yazılım ürün kalitesi modeli büyük olasılıkla ISO/IEC 25010'dur ve 2011'de ISO/IEC 9126'nın yerini almıştır. Bu modelde kalite özellikleri işlevsel uygunluk, performans verimliliği, uyumluluk, kullanılabilirlik, güvenilirlik, güvenlik, bakım yapılabilirlik ve taşınabilirlik gibi başlıklar altında toplanır.

## 13. Yeniden Kullanılabilirlik Nasıl Sağlanır?

Hoca yeniden kullanılabilirliğin pek çok gereği olduğunu söyledi ve öğrencilere "Yeniden kullanılabilirliği sağlamak için ne yapmak gerekir?" diye sordu.

Bir öğrenci şunları önerdi:

- İş kodlarını, yani çekirdek (core) işleri bağımsız tutmak.
- Veri tabanına veri ekleme ve çıkarma kodlarını bağımsız tutmak.
- Loglama (kayıt tutma) ve kimlik doğrulama (authentication) gibi işleri de bağımsız tutmak.

Hoca bunu onaylayarak tasarım boyutuna bağladı: Tasarım sırasında az bağımlılık (low coupling) ve yüksek uyum (high cohesion) sağlanmalıdır. Modüllerin ya da sınıf yapısı çerçevesinde sınıfların bağımsızlığı sağlanmalıdır. Yapılardaki bağımsızlık arttıkça yeniden kullanılabilirlik de artar.

Başka bir öğrenci nesneye dayalı uygulamalarda modülerliği önerdi. Hoca bunu da kabul etti, ama yeniden kullanılabilirliğin "yalnız modülerlik" olmadığını vurguladı. Modülerlik, az bağımlılık ve yüksek uyum birlikte bunu sağlar.

Hoca listeyi şu maddelerle tamamladı:

- **Tasarım kalıpları:** Bir probleme bir kez çözüm getirdiysek bu çözümü defalarca kullanabiliriz.
- **Yazılım kütüphaneleri:** Bir fonksiyonu veya programı yazıp genelleştirerek bir kütüphane hâline getirebiliriz. Bu kütüphane bir şirketin içinde olabileceği gibi bir dilin derleyicisi içinde de olabilir; herkesin kullanacağı genel amaçlı yapılar olabilir.
- **Nesneye dayalı yaklaşım:** Nesneye dayalı yaklaşımın kullanılması zaten baştan yeniden kullanılabilirliği sağlayan bir özelliktir.

Özetle yeniden kullanılabilirliği sağlayan yapılar ya da çözümler şunlardır: modülerlik, az bağımlılık, yüksek uyum, tasarım kalıpları, yazılım kütüphaneleri ve nesneye dayalı yaklaşım.

## 14. Geliştirme Sürecinde Kaliteyi Sağlamak

### 14.1 Aşamalara göre kalite

Hoca yazılım geliştirme sürecinde kaliteyi sağlamak için her aşamada yapılacakları saydı:

- **Gereksinim analizi aşamasında:** Sistemin kolay öğrenilir olmasına çaba gösterilir.
- **Modüllere ayırma aşamasında:** Taşınabilirliğe ve birlikte çalışabilirliğe (bağlanabilirliğe) önem verilir.
- **Ayrıntılı tasarımda:** Doğruluk faktörüne ağırlık verilir. Yani algoritmalar oluşturulurken bizden istenenlerin adım adım yerine getirilip getirilmediğine bakılır.
- **Test aşamasında:** Doğruluk, performans ve diğer faktörlerin ölçülmesine çaba gösterilir.

### 14.2 Hata bulma gideri ve hata bulma olasılığı

Kaliteyi sağlamak için hataların ve eksiklerin hemen bulunup düzeltilmesi gerekir. Hoca kalite denince şu iki grafiğin hiçbir zaman gözümüzün önünden gitmemesi gerektiğini söyledi:

- **Hata bulma gideri:** Yazılım geliştirme süreci ilerledikçe hata bulma gideri artar. Hatayı hemen bulursak daha az para harcarız. Hemen bulamazsak, süreç ilerledikçe daha çok para harcamamız gerekir.
- **Hata bulma olasılığı:** Başta daha yüksektir, zaman geçtikçe azalır.

Hoca öğrencilere bu iki grafiğe bakarak süreç içinde ne yapılması gerektiğini sordu. Bir öğrenci birim testini (unit test) önerdi. Hoca birim testinin yapılacağını, ama bunun ancak geliştirme ilerleyip kodlama yapıldıkça mümkün olduğunu söyledi. İlk evrelerde, yani gereksinim analizi ve tasarım yapılırken de hataları yakalamak gerekir. Bunun için:

- Her kilometre taşında (milestone) gözden geçirme (review) yapılır: analizde gözden geçirme, tasarımda gözden geçirme.
- Geliştirilen belgeler ve kodlar, kod denetimi (code inspection) ile çalıştırılmadan ayrıntılı olarak incelenir.
- Kodlamaya geçildikçe birim testleri yapılır.
- Geliştirmeye başlamadan önce kurallı bir yazılım geliştirme süreç modeli seçilir ve bu modele uyulur. Model her proje için farklı seçilebilir. Önemli olan, o projede hangi modelin kullanılacağına karar verip ondan sonra ilerlemektir. Bu, işi kolaylaştırır, çünkü modelin kendi içinde kontrolleri vardır.

  **Hocanın vurgusu:** Süreç ilerledikçe hata bulma maliyeti artar, hata bulma olasılığı azalır. Bu yüzden hatalar olabildiğince erken, her kilometre taşında yapılan gözden geçirmelerle bulunmalıdır.

### 14.3 Yazılım kalitesinin sağlanma aşamaları

Hoca yazılım kalitesini sağlamanın aşamalarını şöyle sıraladı:

- Geliştirmenin planlanmasında kalite kontrol yöntemlerinin ve araçlarının belirlenmesi.
- Geliştirme sürecinde durak noktalarında yapılan gözden geçirmeler.
- Kaynak programın sınanması. Sınama (test) teknikleri saydam kutu ve kara kutu testi olarak iki çerçevede yapılır.
- Konfigürasyon yönetim sistemi: konfigürasyonun kurulması, değişikliklerin kontrolü ve rapor düzenleme.

Hoca konfigürasyon yönetiminin daha kaliteli yazılım geliştirmeye yol açan bir özellik olduğunu söyledi ve dersin başında CMMI'nın ikinci düzeyini anlatırken bundan söz ettiğini hatırlattı (Bölüm 4.2).

## 15. Test Tekniklerine Kısa Giriş

Hoca yazılım kalitesinin diğer konularına vizeden sonra geçeceğini söyledi ve test hakkında yalnız kısa bir giriş yaptı.

Yazılım test teknikleri denince genelde iki teknik sayılır, ama aslında üçüncü bir tür daha vardır:

- **Saydam kutu testi (white box testing).**
- **Kara kutu testi (black box testing).**
- **Gri kutu testi (gray box testing):** Hocaya göre özellikle sonraki yıllarda gri kutu testi üzerine daha fazla çalışma yapıldı.

**Ek bilgi (derste anlatılmadı):** Saydam kutu testinde test eden, programın iç yapısını (kodu, kontrol akışını) bilerek test durumları tasarlar. Kara kutu testinde iç yapıya bakılmaz; yalnız girdiler ve beklenen çıktılar üzerinden, spesifikasyona göre test edilir. Gri kutu testi ikisinin arasındadır: Test eden iç yapı hakkında kısmi bilgiye (örneğin veri yapıları ya da mimari) sahiptir, ama testi yine dışarıdan, arayüz üzerinden yapar.

Yazılım test süreci dört adımda gerçekleştirilir:

1. Birim testi (unit test).
2. Bütünleştirme testi (integration test).
3. Sistem testi (system test).
4. Kullanıcı kabul testi (user acceptance test).

Hoca bu konuların ayrıntısına vizeden sonra girileceğini söyledi.

## Örnek Sorular

**1. (derste sözlü, 00:06:37)** CASE nedir? Staj yaptığınız yerde "CASE kullanacağız" dendiğinde "CASE nedir?" sorusuna ne cevap verirsiniz?

**Çözüm (derste verildi):** CASE, bilgisayar destekli yazılım mühendisliğidir (Computer-Aided Software Engineering). CASE araçları yalnız UML diyagramlarını hızlı çizmek ve kontrol etmek için değil, yazılım geliştirme sürecinin her evresinde (analiz, tasarım, kodlama, konfigürasyon yönetimi, test) yazılım geliştiricilere destek olan, işlerini hızlandıran ve daha kontrollü yapılmasını sağlayan yazılımlardır. Bütünleşik büyük araçlar ve tek aşamaya yönelik küçük araçlar olarak ikiye ayrılır.

**2. (derste sözlü, 00:14:11)** CMMI nedir? CMMI'nın kaç düzeyi vardır? Hangi sertifikasyonlar alınabilir?

**Çözüm (derste verildi):** CMMI, Capability Maturity Model Integration, yani bütünleşik yetkinlik olgunluk modelidir. Bir yazılım geliştirme birimini süreçlerine, proje yönetimine ve insan kaynağına ilişkin yetkinliklerine göre derecelendirir. Beş düzey vardır. İkinci düzeyde gereksinim analizi yapılır, SRS ve tasarım belgeleri hazırlanır, proje planı yapılır ve konfigürasyon yönetimi uygulanır. Üçüncü düzeyde risk yönetimi yapılır ve bütün projelerde aynı yapıda belgeler kullanılır.

**3. (derste sözlü, 00:17:28)** Yazılım konfigürasyon yönetiminden ne kastediyoruz?

**Çözüm (derste verildi):** Yazılımın her aşamasında elde edilen ürünlerin (belgeler, kod, sürümler) yapısı hakkında bir veri tabanı oluşturulması ve bu yazılım elemanlarının takip edilmesidir. Her sürümde hangi elemanın hangi durumda olduğu tanımlanır. Kalite açısından konfigürasyonun kurulması, değişikliklerin kontrolü ve raporlama adımlarını içerir.

**4. (derste sözlü, 00:20:47)** Türkiye'de CMMI sertifikası alan firmalara örnek verin.

**Çözüm (derste verildi):** Derste Koç Sistem, İTİA ve sürekli beşinci düzeyde olan, askerî yazılım üreten Milsoft örnek verildi. Hocaya göre bu alanda yazılım üreten firmaların en az üçüncü düzeyde olması şartı vardır.

**5. (derste sözlü, 00:24:47)** Tasarım kalıbı (design pattern) nedir? Neden kullanırız?

**Çözüm (derste verildi):** Tasarım kalıbı, sınıf diyagramlarında, sıralı diyagramlarda ve sınıflar üzerindeki işlemlerde daha önce çözümü bulunmuş, verimliliği kanıtlanmış algoritma ve yapıların hazır hâle getirilip sürekli kullanılacak biçimde saklanmasıdır. "Aklın yolu birdir": Bir kez çözülen problemle tekrar uğraşmamak için çözüm kalıp olarak saklanır. Yeniden kullanılabilirliği sağlayan en önemli etkenlerden biridir.

**6. (derste sözlü, 01:25:37)** Yeniden kullanılabilirliği sağlamak için neler yapılması gerekir?

**Çözüm (derste verildi):** İş kodlarını (core), veri tabanı işlemlerini, loglama ve kimlik doğrulama gibi işleri bağımsız modüllere ayırmak; tasarımda az bağımlılık ve yüksek uyum sağlamak; modülerlik; tasarım kalıpları kullanmak; genelleştirilmiş fonksiyonlardan yazılım kütüphaneleri oluşturmak; nesneye dayalı yaklaşımı kullanmak.

**7. (derste sözlü, 01:31:40)** Süreç ilerledikçe hata bulma giderinin arttığını ve hata bulma olasılığının azaldığını gösteren iki grafiği dikkate alarak, yazılım kalitesini sağlamak için geliştirme sürecinde ne yapmamız gerekir?

**Çözüm (derste verildi):** Hatalar olabildiğince erken bulunmalıdır. Her kilometre taşında (analizde, tasarımda) gözden geçirme yapılır; belgeler ve kodlar çalıştırılmadan incelenir (code inspection); kodlamaya geçildikçe birim testleri yapılır; geliştirmeye başlamadan kurallı bir süreç modeli seçilir ve ona uyulur.

**8. (derste sözlü, 01:36:54)** Yazılım test süreci hangi adımlardan oluşur?

**Çözüm (derste verildi):** Dört adımdan oluşur: birim testi, bütünleştirme testi, sistem testi ve kullanıcı kabul testi (user acceptance test).

## Dersin Toparlaması

Bu derste dönemin başından beri işlenen konular vizeden önce bir kez daha gözden geçirildi: CASE araçları, klasik, Unified Process ve çevik süreç modelleri, CMMI düzeyleri ve konfigürasyon yönetimi, gereksinim mühendisliği ve UML şemaları, tasarım kalıpları. IEEE 830 yapısındaki örnek bir SRS belgesi üzerinden belgenin giriş, genel tanım, özel gereksinimler ve ekler bölümleri ile her işlev için hazırlanması gereken kullanım senaryosu belgesi incelendi. Teklif hazırlama ve sipariş alma örnekleriyle, etkinlik şemasının kulvarlar sayesinde hangi kullanıcının hangi işi yaptığını göstermesi ve çatal/birleşme ile paralel akışları ifade etmesi pekiştirildi.

Dersin sonunda yazılım kalitesine geçildi. McCall'ın bakış açılarına göre kalite özellikleri, yeniden kullanılabilirliği sağlayan yapılar ve geliştirme boyunca kalitenin nasıl sağlanacağı anlatıldı. Hataları erken bulmanın önemi, hata bulma gideri ve olasılığı grafikleriyle vurgulandı. Kalite ölçümü, kalitenin sınanması ve saydam, kara ve gri kutu test teknikleri vizeden sonraki haftalarda ayrıntılı olarak işlenecek.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.